Predictive GPU Right-Sizing for Production Online Inference

A Research Proposal Grounded in the Alibaba Serverless Infrastructure GPU Trace

Saksham Adhikari Sudip Bhandari

Advisor: Dr. Fred Zhao

ISAN 3392 • Texas State University

Fall 2026 • Working draft, October 26, 2026

Abstract

Production GPU fleets serving online inference are routinely over-reserved: operators trade utilization for latency headroom because an under-provisioned inference pod hurts users directly. We ask how much of that reserved capacity can be recovered by predicting, at admission, the smallest safe GPU slice a pod needs (*Model A, allocation*), and what placement risk the recovery carries (*Model B, readiness/failure*). Using the six-month Alibaba Serverless Infrastructure (ASI) trace released with OSDI '26 (up to 155,410 GPUs on 37,707 servers), we built two datasets (294,465 pods for right-sizing; 2,826,036 pods for placement) and evaluated history-rule, linear, and gradient-boosted models under a pre-registered go/no-go gate on ten held-out clusters. At a fixed 5% under-provisioning budget, roughly **21–25% of reserved online-inference GPU-hours** are reclaimable on clusters never seen in training, almost entirely from recommendation pods with workload history; GenAI pods have a median minimum viable fraction of 0.983 of a GPU, consistent with ASI's report that GenAI inference fills GPU memory while leaving compute mostly idle. A simple workload-history rule captures nearly all of it (20.8% vs. 21.9% for XGBoost), so the learned model returned NO-GO on its accuracy gate. Model B predicts failure to become ready with ROC-AUC 0.75 in known clusters but 0.65 and poorly calibrated on unseen clusters. This document preserves the full research trail (hypotheses, experiments, failures, and comparisons), positions the results against commercial GPU schedulers at Google, AWS, Microsoft, NVIDIA, Meta, and Alibaba itself, quantifies business value, and proposes the next round of experiments: workload-mixing signals (recommendation vs. GenAI co-location), workload–GPU compatibility, production vs. non-production behaviour, runtime patterns, and explicit safety constraints.

Reading guide

Model A	*Allocation.* Predicts the minimum viable GPU fraction (MVF) per online-inference pod at admission, with a conformal safety margin. Success = reserved GPU-hours reclaimed (R) at under-provisioning rate $U \leq 5\%$.
Model B	*Readiness/failure.* Predicts the probability a pod fails to become ready when placed. Success = discrimination (ROC/PR-AUC) and calibration (ECE) that hold on unseen clusters.
Status tags	DONE = run and reported; PROPOSED = designed here, not yet run. No numbers in this document are reported for PROPOSED experiments.

Contents

1 Motivation and Problem Statement **4**

2 Research Questions and Hypotheses **5**

3 Background: How Commercial GPU Schedulers Optimize Allocation **5**
- 3.1 Alibaba (the operator behind our trace) . . . 5
- 3.2 Google . . . 6
- 3.3 Amazon Web Services . . . 6
- 3.4 Microsoft . . . 6
- 3.5 Meta . . . 7
- 3.6 NVIDIA platform mechanisms . . . 7
- 3.7 Colocation and interference research . . . 7
- 3.8 Prediction of needs and failures . . . 7
- 3.9 Summary of industry approaches . . . 8

4 Data **8**
- 4.1 Source . . . 8
- 4.2 Early exploration (research trail) . . . 8
- 4.3 Modelling datasets . . . 9

5 Method **9**
- 5.1 Model families . . . 9
- 5.2 Evaluation protocol . . . 9

6 Results to Date **10**
- 6.1 Model A: allocation . . . 10
- 6.2 Model B: readiness / failure . . . 11
- 6.3 Combining A and B . . . 12
- 6.4 Answers so far . . . 13

7 Failures, Negative Results, and What They Taught Us **13**

8 Our Setting vs. Industry: Why the Results Differ **14**
- 8.1 Why our numbers differ, and why that is informative . . . 14
- 8.2 Which gaps are addressable, and which are inherent? . . . 15

9 Business Value **15**
- 9.1 Framing the headline number . . . 15
- 9.2 Lower GPU cost . . . 16
- 9.3 Less idle capacity . . . 17
- 9.4 Faster job completion . . . 17
- 9.5 More reliable provisioning . . . 17
- 9.6 Caveats on value . . . 18

10 Proposed Experiments and Feature Engineering **18**
10.1 New feature families 19
10.2 Experiment cards 19
10.3 Experiment summary 22

11 Limitations **22**

12 Plan and Timeline **23**

13 Conclusion **23**

A Research Trail (Chronological) **27**

B Reproducibility **27**

1 Motivation and Problem Statement

Companies serving interactive AI products (feeds, ads, search ranking, chat assistants) prioritize low tail latency because it drives engagement and satisfaction. The standard operational response is to over-reserve: give each inference pod a whole GPU, or more, and accept low utilization as the price of headroom. Because accelerators dominate the capital and energy cost of an AI datacenter, even small utilization gains translate into large savings.

The utilization identity. Suppose a fleet of N servers runs at utilization R_1, and better allocation raises it to R_2 while serving the same work. Ignoring packing and failure constraints, the number of servers X that can be freed satisfies

$$NR_1 = (N - X)R_2 \quad \Longrightarrow \quad X = N\,\frac{R_2 - R_1}{R_2}. \tag{1}$$

With $N = 100{,}000$, $R_1 = 28\%$, $R_2 = 40\%$, we obtain $X = 30{,}000$ servers; at an assumed \$5,000 per server this is \$150M of avoided capital. Figure 1 shows the identity is concave: the first points of utilization are the most valuable.

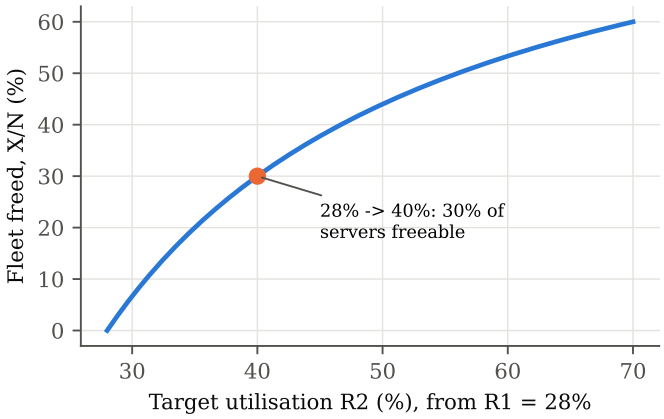


Figure 1: Share of the fleet freed by raising utilization from 28% to R_2 (eq. (1)). Idealized: assumes perfect packing and no added failure risk. Sections 6 and 9 show why the realizable number is smaller.

Research question. *How much GPU capacity in production online-inference fleets can be recovered by predicting, at admission, the smallest safe GPU slice a pod needs, and what placement risk does that recovery carry?* We separate the problem into two models with distinct jobs:

- **Model A (allocation / right-sizing):** a quantile regressor for the pod's minimum viable GPU fraction, wrapped in a split-conformal margin so that at most ≈5% of pods are under-provisioned.
- **Model B (readiness / failure):** a calibrated classifier for whether the pod becomes ready after placement, used to veto shrinks that make placement unsafe.

Keeping these separate matters: Model A answers "how much does this pod need?"; Model B answers "if we give it that, will it come up?" Conflating them hides the second risk behind the first.

2 Research Questions and Hypotheses

Table 1: Research questions, hypotheses, and current status. H1–H4 were pre-registered before modelling; H5–H10 are added in this proposal.

ID	Hypothesis	Model	Status
H1	A learned quantile model (A3) predicts minimum viable slice more accurately than a per-workload P90 history rule (A1) on unseen clusters.	A	DONE: rejected
H2	Failure to become ready is predictable and calibrated from admission-time features.	B	DONE: partly supported
H3	A material share (>10%) of reserved GPU-hours is reclaimable at $U \leq 5\%$.	A	DONE: supported (≈21%)
H4	Relationships transfer to clusters unseen in training.	A, B	DONE: partly
H5	*Workload mixing.* Pods on servers dominated by recommendation workloads have lower, more predictable MVF than pods co-located with GenAI; co-location share improves A's pinball loss and B's PR-AUC.	A, B	PROPOSED
H6	*Compatibility.* Readiness failure depends on the (workload type × GPU type × slice size) interaction, not on each factor alone.	B	PROPOSED
H7	*Production vs. non-production.* High-priority (production) pods have tighter, more stable demand and lower reclaimable share but lower failure; mixing them with low-priority pods raises B's risk for the latter.	A, B	PROPOSED
H8	*Runtime patterns.* Diurnal amplitude and burstiness from early-life telemetry predict MVF better than static request features for cold-start pods.	A	PROPOSED
H9	*Safety constraints.* A per-segment risk budget (tighter for GenAI/production, looser for rec/non-production) reclaims more than a uniform 5% budget at equal expected harm.	A+B	PROPOSED
H10	*Hybrid policy.* History rule for warm pods + A3 for cold-start pods matches or beats either alone.	A	PROPOSED

3 Background: How Commercial GPU Schedulers Optimize Allocation

This section surveys how hyperscalers and platform vendors raise GPU utilization in production, with emphasis on *what signals they use*, *at what granularity they allocate*, and *which constraints bind*. Vendor-reported numbers are labelled as such.

3.1 Alibaba (the operator behind our trace)

ASI, OSDI '26 [12]. The paper accompanying our trace finds that *high GPU demand does not imply high effective utilization*: idle GPUs are stranded across nodes, lack matching CPUs, or violate network locality, and users reserve ample headroom for production safety (diurnal peaks, failover, peak events). Key facts for this project:

- Online inference is 63% recommendation models; classical DNNs (CTR, recommendation, OCR) take ≈70% of online-inference GPU-hours. GenAI is 71% of training and 98% of

offline inference. Over 99% of jobs pin a specific GPU model.

- Online-inference median SM utilization is 6% and median GPU-memory utilization 30%. GenAI inference uses 94% of GPU memory but only 5% of SM; conventional DNN inference uses 20% memory and 6% SM.
- GPU sharing is rare and fractional fragmentation is "now negligible"; fractional requests come almost only from CV and recommendation models using MPS, because MPS/MIG isolation is coarse and GenAI memory footprints are large.
- Deployed mechanisms act *after* reservation: IPC defragmentation reduces nodes with slack by 20.2%; **SpotGPU** lends high-priority jobs' reserved-but-idle capacity to preemptible low-priority jobs, raising the allocation ratio from 68% to 93%, and its preemption-cost-aware victim selection cuts low-priority job completion time by 24%.
- Colocating CPU jobs lowers training SM utilization by 10% (median) and 18% (P90). The paper names GenAI (memory-bound) + DNN (compute-light) colocation as complementary but unrealized, because MIG is "too static" and time-sharing violates SLOs.

Earlier Alibaba systems. The PAI trace study [24] found the median instance uses 0.042 GPU, and that sharing would need only 50% of requested GPUs on average (73% savings at peak); over 75% of CTR instances use less than 0.1 GPU. FGD [25] schedules fractional GPU requests to minimize fragmentation, cutting unallocated GPUs by up to 49% in emulation. AntMan [28] co-designs the scheduler with DL frameworks to raise memory and compute utilization by 42% and 34%. Prism [30] disaggregates CPU and GPU parts of DLRM serving, cutting GPU fragmentation by 27%. Aegaeon [26] pools GPUs across many LLMs with token-level model switching, reporting a reduction from 1,192 to 213 H20 GPUs (≈82%) in a three-month beta.

3.2 Google

Autopilot [19] vertically right-sizes Borg tasks from 5-minute usage histograms, using decayed percentile recommenders (e.g. P98 or max for memory depending on OOM tolerance) plus a 10–15% safety margin, and an ML recommender that trades overrun against underrun cost. It cut slack to 23% (ML) and 31% (moving window) versus 46–60% for manually sized jobs and reduced severe OOMs 10×. Kubernetes' Vertical Pod Autoscaler descends from it. For GPUs, **GKE** [8] offers three sharing strategies: MIG (hardware isolation, up to 7 slices), time-sharing (no memory isolation between clients), and MPS (limited isolation), selected through node labels.

3.3 Amazon Web Services

SageMaker inference components [1] let many models share an endpoint with per-model accelerator allocations, scale-to-zero, and utilization-aware placement; AWS reports 50% lower deployment cost on average (vendor claim). On **EKS** [3], GPU sharing uses NVIDIA time-slicing (no memory isolation) or MIG (hardware isolation, but re-partitioning requires a GPU reset). Karpenter counts every `nvidia.com/gpu` request as a full GPU even under time-slicing, which limits autoscaled sharing.

3.4 Microsoft

The **Philly** study [11] reported ≈52% average utilization of in-use GPUs, driven by locality/gang scheduling and colocation interference, and ≈30% of jobs failing, mostly early. **Gandiva** [27] ex-

ploits training-iteration predictability for time-slicing and migration. **Singularity** [22] provides transparent, checkpoint-based preemption, migration and elastic resizing across a global fleet. **Resource Central** [5] is the closest analogue to Model A: it predicts VM utilization buckets from admission-time attributes and subscription history (79–90% accuracy) and drives safe oversubscription in Azure. An ICSE '24 study [7] found most low-GPU-utilization jobs are fixable with small code changes. AKS exposes MIG node pools with profiles fixed at creation [13].

3.5 Meta

Meta's ads/DLRM inference fleet has a long tail of low-utilization models caused by tail-latency provisioning [20]. Meta chose hardware multi-tenancy (static, reconfigurable only when idle) over software multi-tenancy because a single fault kills every process on a shared GPU, and sizes containers from per-model load tests.

3.6 NVIDIA platform mechanisms

MIG [16] partitions an H100 80GB into fixed profiles (e.g. 7×1g.10gb, 3×2g.20gb, 2×3g.40gb). **MPS** partitions SMs and memory in software. The open-source **KAI Scheduler** (from Run:ai) adds fractional GPU allocation and queue-based fairness to Kubernetes. **Dynamo** [15] disaggregates LLM prefill and decode with dynamic GPU scheduling.

3.7 Colocation and interference research

Gavel [14] uses per-job, per-accelerator throughput matrices; REEF [9] offers microsecond preemption so real-time inference can share with best-effort work; Orion [23] schedules at operator level using each kernel's compute- vs. memory-bound profile; Usher [21] estimates model resource needs from kernels and places to avoid interference; MuxServe [6] co-locates LLMs by popularity, separating prefill and decode; iGniter [29] models inference latency under colocation. The signals that predict interference are consistent across these papers: compute vs. memory-bandwidth intensity, memory/KV-cache footprint, request burstiness, host-CPU contention, and model popularity.

3.8 Prediction of needs and failures

Beyond Autopilot and Resource Central, MagicScaler [17] uses uncertainty-aware forecasts with stochastic QoS constraints, and Helios [10] uses history-based duration prediction for scheduling. Conformal prediction [4] and conformalized quantile regression [18] provide distribution-free coverage; we found no peer-reviewed application to GPU slice sizing, which is a gap this project fills. **Closest prior work to Model B:** Zhang and Liao [31] (arXiv, 18 Oct 2026) study "ineffective occupancy" on an Alibaba GPU trace: 52.70% of execution records never became ready; a gradient-boosted classifier on admission attributes reaches ROC-AUC 0.8187 under machine-grouped 5-fold CV; and readiness-aware scheduling cuts ineffective occupancy from 43.64% to 27.15%. The trace version is not stated on the abstract page.

How this project differs from Zhang and Liao. (i) Our Model B is evaluated on *unseen clusters* (LOCO) and on *future days*, not grouped CV within a pool, which explains a lower AUC (0.65–0.75 vs. 0.82) and exposes the calibration-transfer problem; (ii) we score calibration (ECE), not only discrimination; (iii) Model B is used as a *veto* on Model A's shrink decisions, so the joint allocation-and-readiness question is new; (iv) our population is restricted to online inference (rec + GenAI). A direct replication of their protocol on D2 is a cheap robustness check (added to E8).

3.9 Summary of industry approaches

Table 2: Commercial and production GPU allocation approaches.

System	Signals	Granularity	Key constraint	Reported effect
Alibaba ASI SpotGPU	Reserved vs. used, priority, checkpoint age	Whole job / GPU	Preemptible LP only; HP untouched	Allocation 68%→93%
Alibaba Aegaeon	Per-model request rate, tokens	Token-level model switching	LLM SLOs	1,192→213 GPUs
Alibaba FGD	Fractional requests, node state	Fractional GPU	Fragmentation	−49% unallocated
Google Autopilot	5-min usage histograms	CPU/RAM limits	OOM tolerance, 10–15% margin	Slack 46–60%→23%
GKE / EKS / AKS	Admin-chosen strategy	MIG / MPS / time-slice	Isolation, static profiles	–
AWS SageMaker IC	Per-model traffic, utilization	Per-model accelerator share	Endpoint SLO	−50% cost (vendor)
Azure Resource Central	Admission attributes, history	VM utilization buckets	Safe oversubscription	79–90% accuracy
Meta inference	Per-model load tests	Container "T-shirt" sizes	Fault isolation	"Appreciable"
NVIDIA MIG / KAI / Dynamo	Profiles, queues, phase	Up to 7 slices; prefill/decode	Fixed profiles	–
This project	Request, history, cluster load, occupancy	27-size trace grid	$U \leq 5\%$, readiness risk cap	≈21–25% reserved GPU-h (opportunity)

4 Data

4.1 Source

The data come from the anonymized Alibaba Serverless Infrastructure (ASI) GPU trace (`cluster-trace-gpu-v2026`) released with the OSDI '26 characterization paper [12]. It covers about six months of a hyperscale production AI cluster serving heterogeneous workloads (ad-hoc development, training, online inference, offline inference) and, at the hourly peak, 155,410 GPUs across 37,707 GPU servers. The trace provides fact tables for workload mix, job and model types, priority classes, resource requests and utilization, GPU and server inventory, access-switch (ASW) topology, network traffic, and execution time.

4.2 Early exploration (research trail)

The project began from an open question ("can scheduling patterns in GPU traces raise utilization without hurting inference SLAs?") and an explicit list of unknowns: the schema, a tractable working sample, a research question that fits one semester, and storage/compute budget. Initial exploratory visualizations used a 1–2% sample of the trace. A supervisor comment narrowed the scope to *recommendation + GenAI data for online inference*, which became the D1/D2 population below. Distributed-training topics (e.g. FSDP) and a Kaggle-style training example were set aside until the business problem was fixed.

Data repair. Day and hour fields were missing on ingest; they were recovered from source file paths before any dataset was built. This is recorded because every temporal split depends on it.

4.3 Modelling datasets

Table 3: Modelling datasets built from the ASI trace.

Dataset	Used for	Rows (pods)	Target
`D1_RIGHTSIZING`	Model A	294,465	Y_{MVF}: smallest GPU fraction covering the pod's busy hours (median 0.231 rec, 0.983 GenAI)
`D2_PLACEMENT`	Model B	2,826,036	Y_{READY}: whether the pod became ready at admission (failure rate 46%)

Scope. Online-inference pods for recommendation (rec) and GenAI models with a GPU request. D1 additionally requires whole-GPU pods observed for at least 24 hours. Between 61% and 68% of pods have no workload history (cold start).

5 Method

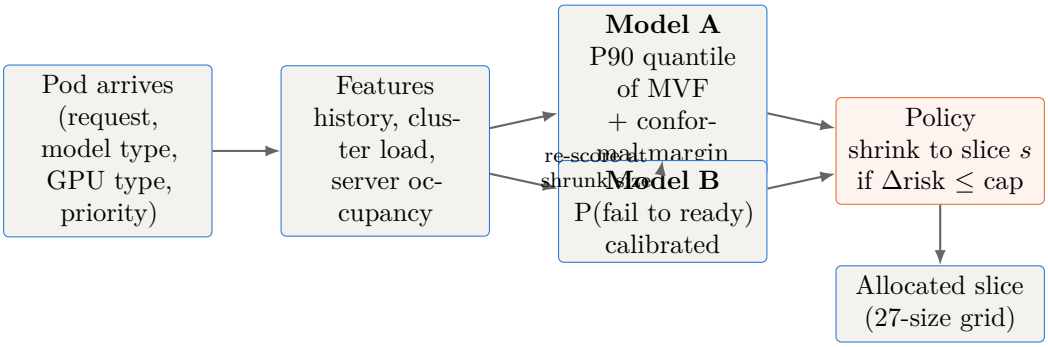


Figure 2: Two-model framing. Model A proposes a smaller slice; Model B re-scores placement risk at that size; the policy accepts the shrink only if the predicted increase in failure risk is under a cap.

5.1 Model families

Table 4: Model families. Each family received the same tuning budget (three configurations, chosen on validation only).

Model	Description
A1	Per-workload 90th percentile of past need over the last 7 days (only pods finished before admission); falls back to the model-type × GPU-type percentile.
A2	Regularized linear quantile regression ($\tau = 0.90$).
A3	XGBoost quantile regression ($\tau = 0.90$) with monotone constraints on history features.
B1 / B2 / B3	Logistic regression / random forest / LightGBM classifiers for failure to become ready, with Platt or isotonic calibration.

5.2 Evaluation protocol

- **Temporal split.** Train days 14–129; validation days 130–157 (split by workload into an early-stopping half and a calibration half); test days 158–183.

- **Leave-one-cluster-out (LOCO).** For each of the 10 clusters with ≥9,000 pods, train on the other clusters' training days and score that cluster's test days.
- **Safety margin.** Split-conformal margin per (model type × cold/warm) targeting 95% coverage, then rounded *up* to one of the 27 slice sizes observed in the trace.
- **Metrics.** Pinball loss ($\tau = 0.90$); U = share of pods given less than they needed; R = share of reserved GPU-hours reclaimed; for Model B, PR-AUC, ROC-AUC, Brier, ECE; paired Wilcoxon signed-rank tests across the 10 clusters.
- **Pre-registered gate.** A3 passes only if it beats A1 on pinball loss in ≥8 of 10 clusters with Wilcoxon $p \leq 0.05$, *and* reclaims more while keeping U near 5%.

6 Results to Date

6.1 Model A: allocation

Table 5: Model A on unseen clusters (mean of 10 held-out clusters, test days) and on seen clusters (future test days).

	Unseen clusters (LOCO)				Seen clusters		
Model	Pinball	R	U	Coverage	Pinball	R	U
A1 history rule	**0.0247**	11.0%	**0.7%**	**96.7%**	0.0249	8.7%	**0.5%**
A2 linear QR	0.0266	12.9%	1.4%	90.0%	0.0266	11.1%	1.1%
A3 XGBoost QR	0.0259	**14.3%**	2.0%	92.2%	**0.0237**	**13.0%**	1.0%

Table 6: Pre-registered go/no-go gate for A3 vs. A1 on unseen clusters.

Criterion	Result	Outcome
A3 more accurate in ≥8 of 10 clusters	4 of 10	**Fail**
Wilcoxon $p \leq 0.05$ on pinball loss	$p = 0.81$	**Fail**
More reclaimed with U near 5%	14.3% vs. 11.0%, $p = 0.007$	**Pass**
Decision		**NO-GO**

Matched-risk comparison (exploratory, added after the gate). Holding every model to exactly $U = 5\%$ in each cluster, A3 is ahead in 8 of 10 clusters by a mean of 1.1 points (median 0.7), reclaiming 21.9% vs. 20.8% for A1; this is not significant (Wilcoxon $p = 0.14$). The history rule wins clearly in clusters `3f39` and `7087`. Figure 3 shows the picture: A3's apparent advantage at default margins is mostly bought risk.

Where savings come from. Almost all reclaimable capacity sits in recommendation pods with workload history (fig. 4). A3's one clear advantage is on recommendation pods *without* history (8.1% vs. 0%), the cold-start case, which motivates the hybrid policy (H10).

Other Model A findings.

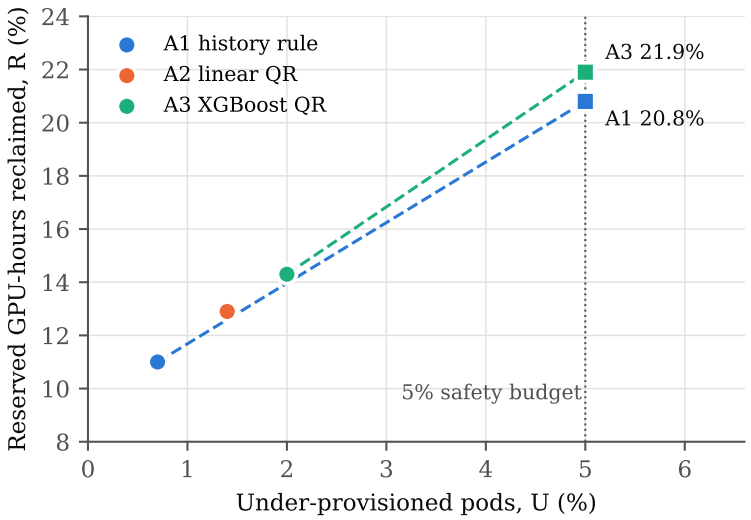


Figure 3: Reclaim–risk trade-off on unseen clusters. Circles: default conformal margins. Squares: margins re-tuned so each model sits at exactly $U = 5\%$. Dashed lines connect the two operating points of the same model and are not measured curves.

- **Slice sizes cap savings.** The 27 sizes in the trace jump from 0.5 to 0.7 to 1.0 GPU, so any pod needing more than 0.7 keeps a whole GPU (fig. 5). This alone explains why GenAI pods (median need 0.983) offer almost nothing.
- **Monotone constraints are free.** Removing them changed pinball loss by 0.0000; an audit confirmed no prediction ever decreases as past demand rises.
- **History and cluster load both help.** Removing workload history raised pinball loss by 0.0045 and cut R by 1.4 points; removing cluster-load features pushed U to 4.5%.
- **Identity features add nothing.** Cluster and switch IDs did not help; separate rec and GenAI models did not beat one pooled model.
- **Sign flips by segment.** `cpu_per_gpu` has Spearman $+0.14$ with need for rec and -0.47 for GenAI, so the originally planned $+1$ monotone constraint would have been wrong.
- **Drivers.** Model type and the workload's past maximum need dominate (SHAP and permutation importance); a depth-3 rule tree reproduces A3 with $R^2 = 0.887$.
- **Latency.** A3 scores one pod in 3.6 ms median, 5.3 ms p95, well inside an admission-path budget.

6.2 Model B: readiness / failure

- Paired tests over 10 unseen clusters: B3 vs. B1 on PR-AUC, better in 6 of 10, $p = 0.065$; on ECE, $p = 0.50$; B3 vs. B2, $p = 0.78$. None significant.
- **Why calibration does not transfer:** cluster failure rates range from 35% to 67%; a model trained elsewhere cannot know a new cluster's base rate.
- Medium risks are underestimated: pods scored at 40% fail about 50% of the time.

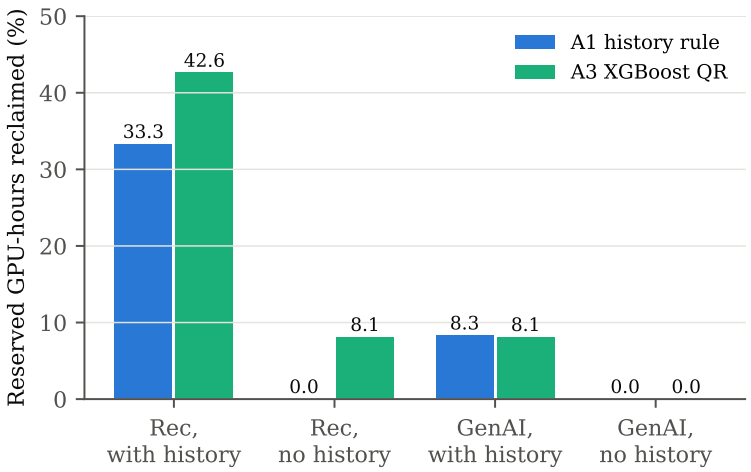


Figure 4: Reserved GPU-hours reclaimed by pod group (seen clusters, test days).

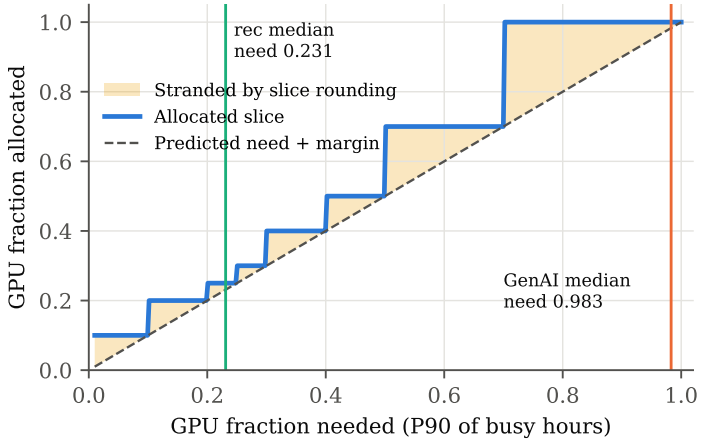


Figure 5: Why the slice grid caps savings. *Illustrative*: the intermediate steps are schematic; the 0.5→0.7→1.0 jumps and the segment medians are from the trace. Any pod whose need-plus-margin exceeds 0.7 is rounded to a whole GPU.

- **Drivers:** priority class, GPU memory requested, GPU type, past compute use, and server occupancy. Big, low-priority pods asking for scarce GPU types on busy servers fail more often.
- **Latency:** 3.0 ms median, 7.8 ms p95.

6.3 Combining A and B

Shrinking pods with Model A and re-scoring them with Model B reclaims 11.4% of long-running pods' GPU-hours, but fractional-GPU pods make up only 0.3–3.7% of each cluster, consistent with the OSDI '26 finding that GPU sharing is rarely used. The composition is therefore reported as a *what-if*, not a validated policy.

Table 7: Model B discrimination and calibration. Failure base rate 46% (PR-AUC baseline ≈0.46). Seen-cluster ECE is after calibration (B3 uses out-of-fold isotonic); unseen PR-AUC uses raw scores.

	Seen clusters			Unseen clusters		
Model	ROC-AUC	PR-AUC	ECE	ROC-AUC	PR-AUC	ECE
B1 logistic	0.640	0.578	0.067	0.575	0.549	0.123
B2 random forest	**0.754**	0.708	**0.044**	**0.671**	**0.635**	**0.112**
B3 LightGBM	**0.754**	**0.717**	0.053	0.651	0.615	0.121

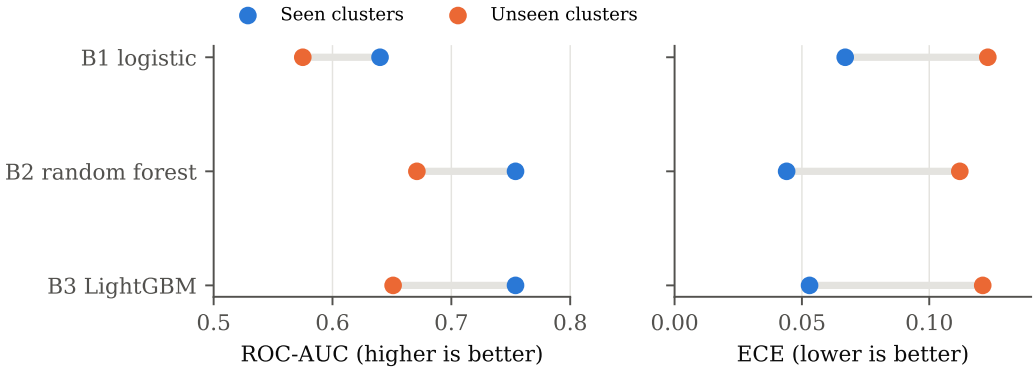


Figure 6: Model B transfer gap. Every model loses discrimination and roughly doubles its calibration error on unseen clusters.

6.4 Answers so far

7 Failures, Negative Results, and What They Taught Us

Negative results are kept in full here because they are the most transferable part of the work.

Table 10: Failure log.

What failed	Evidence	Lesson / response
Learned model vs. history rule (H1)	A3 better in 4/10 unseen clusters, $p = 0.81$; gate NO-GO.	Inference demand is dominated by per-workload persistence. A simple rule is the right production baseline; ML earns its keep only where history is missing (cold start).
Early research question	First framing ("increase utilization without compromising SLA") had no latency field to measure SLA against.	Reframed around U (under-provisioning rate) as an observable safety proxy; latency stays a stated limitation.
Missing timestamps	Day/hour fields absent on ingest.	Recovered from file paths; added a data-validation step before any split.

What failed	Evidence	Lesson / response
History-feature leakage	Building `wl_hist_*` from pods *admitted* on earlier days gave 8% lower test error than using only pods *finished* before admission.	Exported history columns probably carry future information. Rebuild in SQL with a strict finished-before-admission cut (section 10, E9).
Planned monotone constraint on `cpu_per_gpu`	Spearman +0.14 (rec) vs. −0.47 (GenAI).	Domain priors must be checked per segment before they become constraints.
Segment-specific models	Separate rec/GenAI models did not beat a pooled model.	Model type is already a dominant split feature; pooling is simpler and as good.
Cluster/switch ID features	No gain.	Identity features do not generalize; use cluster *state* (load, occupancy) instead.
Model B calibration transfer	ECE 0.04–0.07 seen →0.11–0.12 unseen; base rates 35–67% by cluster.	Needs per-cluster base-rate recalibration (E8) or a hierarchical prior.
Boosting vs. random forest for B	B3 vs. B2 $p = 0.78$.	Model class is not the bottleneck; features and support are.
A+B composition	Only 3.6–12% of shrunk sizes appear in training.	Model B cannot score counterfactual sizes it never saw. Needs either more fractional-GPU data or a simulator (Tier 3).
Conformal coverage on unseen clusters	92.2% vs. 95% target for A3.	Exchangeability breaks across clusters; per-cluster or weighted conformal is needed.

8 Our Setting vs. Industry: Why the Results Differ

8.1 Why our numbers differ, and why that is informative

1. **We measure opportunity; industry reports realized savings.** Autopilot's 46–60%→23% slack reduction and SpotGPU's 68%→93% allocation are outcomes of systems that can act. Our 21–25% is an *upper bound on reclaimable reservation* at a stated risk, before packing and actuation losses. The two answer different questions and should not be ranked against each other.

2. **Different lever, same headroom.** SpotGPU harvests reserved-but-idle capacity *after* reservation by lending it to preemptible jobs; our Model A shrinks the reservation *at admission*. They target the same idle headroom from opposite ends, so our number partly overlaps SpotGPU's gain. This positions right-sizing as complementary (it reduces how much must be harvested and evicted) rather than as a competitor.

3. **The slice grid, not the predictor, is the binding constraint.** Industry gets finer control (MIG sevenths, MPS percentages, token-level switching). In ASI, sharing is rare because isolation is weak, so the trace contains almost no fractional allocations to learn from. This explains both the GenAI zero and Model B's extrapolation in table 8.

Table 8: A+B composition on test-day pods requesting ≥1 GPU and observed ≥24 hours ($n = 55{,}636$).

Risk cap per pod	Pods shrunk	GPU-hours reclaimed	Predicted failure risk	Shrunk sizes seen in training
None	27.5%	11.4%	36.0% →37.8%	12.0%
+10 points	20.3%	7.3%	36.0% →36.5%	8.1%
+5 points	15.2%	5.3%	36.0% →36.1%	5.5%
+2 points	10.5%	3.4%	36.0% →35.9%	3.6%

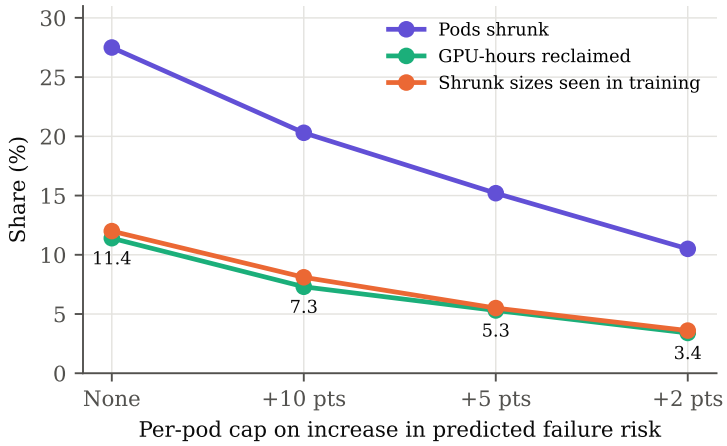


Figure 7: The tighter the risk cap, the fewer shrunk pods land on a slice size Model B has ever observed. Under the +2-point cap only 3.6% do, so the "safe" shrinks are mostly extrapolation.

4. **GenAI is memory-bound, not compute-bound.** ASI reports GenAI online inference at 94% GPU memory but 5% SM. Our median GenAI MVF of 0.983 is consistent with a *memory-dominated* label. A GenAI pod cannot be shrunk by memory, but its idle SMs are exactly what a compute-light recommendation pod could use. This turns the GenAI "zero" into the motivation for the workload-mixing experiment (E1) rather than a dead end. *To verify: which resource term dominates our MVF label per segment.*

5. **Simple rules win when demand is persistent.** Autopilot's production recommender is itself a decayed percentile with a margin, essentially our A1. Our NO-GO result matches industry practice: learned models add most where history is missing.

6. **Generalization is tested harder here.** Industry models are retrained per cluster continuously; we test on clusters never seen in training. The calibration drop for Model B (ECE 0.05→0.12) is the price of that stricter test, and it quantifies how much per-cluster recalibration (E8) is worth.

8.2 Which gaps are addressable, and which are inherent?

9 Business Value

9.1 Framing the headline number

The current result is that **about 21–25% of reserved online-inference GPU-hours** are reclaimable while keeping under-provisioning at or below the **5% safety constraint** ($U \leq 5\%$). The

Table 9: Answers to the pre-registered research questions.

Question	Answer
RQ1: Can the minimum viable slice be predicted better than the percentile-over-history heuristic?	**No** on unseen clusters: A3 better in 4 of 10, $p = 0.81$ (NO-GO). At matched 5% risk, 21.9% vs. 20.8% reclaimed, $p = 0.14$.
RQ2: Is failure to become ready predictable and calibrated?	**Yes** within known clusters (ROC-AUC 0.75, ECE 0.05); weaker and miscalibrated on unseen clusters (ROC-AUC 0.65, ECE 0.12).
RQ3: How much reserved capacity is reclaimable at a fixed risk?	About **21%** of reserved GPU-hours at $U = 5\%$ in the D1 population, almost all from recommendation pods with history. The placement-risk side cannot yet be estimated reliably.
RQ4: Do relationships transfer to unseen clusters?	**Partly**: accuracy roughly holds, but conformal coverage falls to 92% (target 95%) and Model B's probabilities do not transfer.

21% end is the measured matched-risk result on unseen clusters (20.8% A1, 21.9% A3); the 25% end is the upper end of the current estimate range and should be treated as provisional until E9 removes the history-feature leakage. Three tiers keep the claim honest:

9.2 Lower GPU cost

Per 1,000 reserved GPUs there are $1{,}000 \times 8{,}760 = 8.76$M GPU-hours per year. Reclaiming a share r frees $r \times 8.76$M GPU-hours, or equivalently $r \times 1{,}000$ GPUs of steady capacity. We price this at three illustrative on-demand rates: ≈\$1/GPU-h for A10-class cards (approximate; ASI's online fleet is dominated by A10, L20 and H20 parts), \$3.43 for A100 80GB (AWS p4de per GPU), and \$3.90 for H100 (AWS P5 after the June 2025 price cut [2], from secondary aggregators). Owned fleets have lower effective rates than on-demand, so these are upper-range translations.

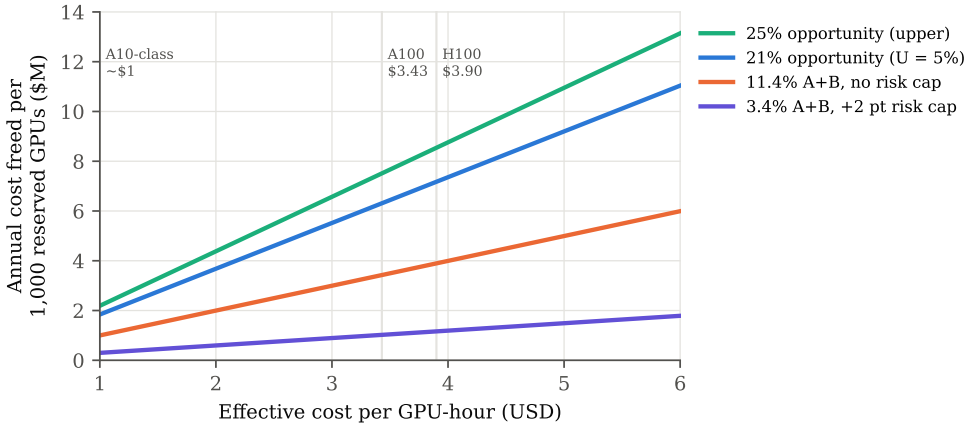


Figure 8: Annual cost freed per 1,000 reserved GPUs as a function of effective GPU-hour price, for each savings tier.

For capital planning, eq. (1) gives the same story: reclaiming 21% of reservations is equivalent to raising effective utilization of the reserved pool by a factor of $1/(1 - 0.21) \approx 1.27$, so a fleet sized for today's demand could absorb ≈27% more of the same workload before buying GPUs.

Table 11: Side-by-side comparison of a production scheduler's vantage point with what the ASI trace exposes to this project.

Dimension	Production schedulers (Google, AWS, Azure, Meta, Alibaba internal)	This project (ASI public trace)
Available features	Live per-request latency, queue depth, model identity, kernel profiles, traffic forecasts, load-test results	Requests, priority, GPU type, model type (anonymized), 20-s utilization telemetry, server/switch topology. **No request latency.**
Workload granularity	Per request / token (Aegaeon, Dynamo), per kernel (Orion, REEF), per container	Per pod, labelled by hourly-P90 busy-hour demand
Allocation granularity	MIG profiles, MPS percentages, time-slices, live resize	27 slice sizes as observed; 0.5→0.7→1.0 jumps; fractional pods only 0.3–3.7% per cluster
Compute scale	10^4–10^5+ GPUs, continuous	Up to 155,410 GPUs, but a 294k-pod (A) / 2.8M-pod (B) modelling slice; 10 LOCO clusters
Scheduler constraints	Isolation, gang scheduling, topology, GPU-model pinning, preemption policy	Same constraints are *in* the data but not controllable; >99% of jobs pin a GPU model
Production requirements	Hard SLOs, A/B rollout, fault isolation, rollback	Safety proxy $U \leq 5\%$; readiness failure as placement risk
Actuation	Scheduler can act and observe the consequence	Observational only: no counterfactual sizes

9.3 Less idle capacity

ASI reports online-inference median SM utilization of 6% and an allocation ratio of 68% for high-priority work before SpotGPU [12]. Right-sizing at admission shrinks reservations *before* they become idle headroom. Because recommendation pods hold most of the reclaimable capacity (33–43% for rec-with-history), the most direct win is consolidating recommendation serving onto fewer GPUs.

9.4 Faster job completion

Capacity freed from over-reserved inference does not disappear; it returns to the pool used by preemptible training, offline inference and development jobs. In ASI, low-priority jobs already run on harvested capacity, and making that capacity *reserved-free* rather than *borrowed* reduces evictions; ASI's preemption-cost-aware victim selection alone cut low-priority completion time by 24% [12]. We do not estimate a JCT gain from the trace; the simulator (E10) would.

9.5 More reliable provisioning

Model B addresses a separate cost: 46% of D2 pods fail to become ready, and each failed placement temporarily occupies GPUs. Zhang and Liao [31] show readiness-aware ordering cuts ineffective occupancy from 43.64% to 27.15% on an Alibaba trace. A calibrated Model B used as a veto keeps right-sizing from trading GPU-hours for failed placements, and used on its own could steer risky pods (large, low-priority, scarce GPU type, busy server) to safer placements.

Table 12: Gap taxonomy. *Modelling* and *feature engineering* gaps can be closed within this project; *inherent* gaps are properties of a public, anonymized, observational trace and are reported, not fixed.

Gap vs. industry	Type	What we can do	Exp.
Leaky history features	Modelling	Strict finished-before-admission rebuild.	E9
Hourly-P90 labels miss peaks	Modelling	P99/max labels from 20-s telemetry; separate memory (hard) and compute (soft).	E9, E5
Calibration does not transfer	Modelling	Prior-shift recalibration; weighted conformal.	E8
Uniform 5% safety budget	Modelling	Segment- and priority-aware budgets.	E5
Cold-start pods (61–68%)	Feature eng.	Early-runtime features; resize after warm-up; hybrid routing.	E4, E6
No co-location / interference signal	Feature eng.	Server/ASW workload-mix features; segregate vs. interleave what-if.	E1
Hardware heterogeneity	Feature eng.	Compatibility cells; throughput-normalized need.	E2
Production tiers	Feature eng.	Priority-class segmentation and interactions.	E3
Coarse 27-size slice grid	Inherent (boundable)	What-if re-rounding gives an upper bound only.	E7
No request latency / SLO field	Inherent	Use U as proxy; state clearly that SLA neutrality is not shown.	–
No counterfactual placements	Inherent (partly)	Simulator replay; flag A+B results as what-if.	E10
No actuation (cannot resize live)	Inherent	Estimate opportunity, not realized savings.	–
Anonymized model identities	Inherent	Use model-type and workload IDs only.	–
One operator, six months	Inherent	LOCO across 10 clusters; Tier-2 transfer if time permits.	E10

9.6 Caveats on value

Realized savings will be lower than the opportunity tiers because (i) freed fractions must be packable onto shared GPUs, and ASI shows sharing is rare due to weak isolation; (ii) U is not a latency SLO; (iii) the A+B tiers rely on Model B extrapolating to unseen sizes. The value table should be read as the ceiling that justifies building the mechanism, not as a forecast.

10 Proposed Experiments and Feature Engineering

Every experiment below is PROPOSED: designed, with a hypothesis and a decision rule fixed in advance, but not yet run. All keep the same splits, LOCO protocol, conformal margins and metrics as section 5, so results remain comparable to tables 5 and 7. Each is tagged by the model it informs: **A** (allocation), **B** (readiness/failure), or **A+B** (policy).

Table 13: Savings tiers, from opportunity to risk-constrained policy.

Tier	Reclaimed	What it assumes
Opportunity (upper)	25%	Provisional upper estimate; Model A only; perfect packing of freed slices.
Opportunity (measured)	21%	Matched $U = 5\%$ on unseen clusters; Model A only; perfect packing.
A+B policy, no risk cap	11.4%	Long-running pods; Model B re-scores each shrink; risk rises 36.0%→37.8%.
A+B policy, +2-point cap	3.4%	Only shrinks Model B deems near-risk-neutral; mostly extrapolated sizes.

Table 14: Annual value per 1,000 reserved inference GPUs. GPU-hours in millions; dollars in millions per year. Scale linearly with fleet size.

Tier	Reclaimed	GPU-h freed	GPUs freed	@$1.00	@$3.43	@$3.90
Opportunity (upper)	25%	2.19	250	$2.19	$7.51	$8.54
Opportunity (measured)	21%	1.84	210	$1.84	$6.31	$7.17
A+B, no cap	11.4%	1.00	114	$1.00	$3.43	$3.89
A+B, +2-pt cap	3.4%	0.30	34	$0.30	$1.02	$1.16

10.1 New feature families

10.2 Experiment cards

E1. Workload mixing: recommendation grouped vs. mixed with GenAI A, B

Hypothesis. (H5) Rec pods admitted to servers whose GPUs are predominantly held by other rec pods have lower and more predictable MVF than rec pods co-located with GenAI or training. Co-location features reduce A's pinball loss and raise B's PR-AUC.

Design. (i) Stratify D1 rec pods into *rec-homogeneous* ($\geq$80% of the server's allocated GPUs are rec), *GenAI-mixed* ($\geq$20% GenAI), and *other-mixed*. Compare MVF distributions, within-workload variance, and A1 pinball loss by stratum. (ii) Add the server/ASW mix features of table 15 to A3 and B3; ablate. (iii) Packing what-if: bin-pack each test day's predicted slices under two placement rules (*segregate*: rec with rec, GenAI with GenAI; *interleave*: first-fit across both) and count GPUs freed.

Metric & decision rule. Mix features are kept if they improve pinball loss in $\geq$7 of 10 LOCO clusters (Wilcoxon $p \leq 0.05$) without raising U, or improve B's PR-AUC by $\geq$0.02. The packing what-if is reported as an upper bound.

Why it matters. Industry schedulers co-locate latency-critical and best-effort work deliberately (section 3). If mixing raises variance, segregation is a free policy lever; if it does not, operators can interleave and pack tighter.

Feasibility. Addressable with the trace's per-server allocation tables. Caveat: in ASI most pods hold whole GPUs, so "mixing" is mostly *host-level* (CPU, PCIe, NIC) rather than SM-level sharing. The result says nothing about true MPS/MIG interference.

Table 15: Candidate features. All are computed strictly from information available at admission time t_0 (or at $t_0 + k$ hours for E4).

Family	Definition (at admission)	Model	Exp.
Server workload mix	Share of the target server's allocated GPUs held by rec / GenAI / training / offline-inference / dev pods; Shannon entropy of that mix; count of distinct workload types.	A, B	E1
Switch (ASW) mix	Same shares aggregated over the access switch; captures network and host contention beyond one server.	A, B	E1
Same-type co-tenancy	Number of pods of the *same workload* already on the server (replica spreading vs. stacking).	B	E1
Compatibility	Requested GPU memory / GPU-type memory capacity; GPU-type scarcity = free GPUs of that type in the cluster / requests pending; historical failure rate of (model type × GPU type × slice) cell, hierarchically smoothed.	B (A)	E2
Normalized need	MVF rescaled by GPU-type throughput so need is comparable across generations.	A	E2
Priority / production	Priority class (production vs. best-effort / spot); preemptible flag; tenant's share of high-priority pods.	A, B	E3
Runtime shape	From the first k hours of telemetry: peak-to-mean ratio, coefficient of variation, 24-h autocorrelation (diurnal strength), slope.	A	E4
Memory headroom	Max / P99 of GPU-memory use (vs. P90 of utilization) to separate hard (OOM) from soft (slowdown) constraints.	A	E5, E9

E2. Workload–pod–GPU compatibility B (A)

Hypothesis. (H6) Readiness failure is driven by the interaction (workload type × GPU type × requested size), especially requests for scarce GPU types, more than by any main effect. GPU-normalized need makes MVF transfer better across clusters with different hardware mixes.

Design. Build a compatibility matrix of failure rate by cell with empirical-Bayes smoothing; add cell rate, memory-fit ratio and scarcity to B; add throughput-normalized need as an alternative A target.

Metric & decision rule. B: unseen-cluster PR-AUC and ECE vs. table 7. A: unseen-cluster coverage closer to 95% than 92.2%.

Why it matters. Mirrors how commercial schedulers filter by node affinity, GPU product labels and MIG profiles before scoring.

Feasibility. Addressable. Risk: sparse cells; smoothing and a minimum-count rule are mandatory.

E3. Production vs. non-production behaviour A, B

Hypothesis. (H7) Production (high-priority) pods have tighter, more persistent demand, lower failure, and a smaller reclaimable share; low-priority pods fail more when placed on servers with many production pods.

Design. Segment by priority class. Report MVF median/IQR, A1 vs. A3 reclaim at $U = 5\%$, and B failure rate per segment. Fit per-segment conformal margins. Interaction test for (pod priority $\times$ server production share) in B.

Metric & decision rule. Report per-segment R at matched U; the interaction is kept if its permutation importance is in B's top 10 on unseen clusters.

Why it matters. Production requirements differ: a GenAI chat endpoint and a best-effort batch scorer should not carry the same safety budget (feeds E5).

Feasibility. Addressable: priority classes are in the trace. Limitation: priority is a proxy for "production"; no explicit SLO tier.

E4. Runtime patterns and a second decision point A

Hypothesis. (H8) For cold-start pods (61–68% of the population), the first $k \in \{1, 6, 24\}$ hours of telemetry predict MVF far better than admission-time features, so a *resize-after-warm-up* policy reclaims more than admission-time sizing.

Design. Add runtime-shape features (table 15) computed on $[t_0, t_0+k]$; predict MVF over the remaining lifetime. Compare reclaim at $U = 5\%$ for admission-only vs. k-hour resize, charging the first k hours at the original size.

Metric & decision rule. Net GPU-hours reclaimed vs. admission-only A3, by k.

Why it matters. This is how vertical autoscalers in industry work (observe, then resize). It directly attacks the cold-start gap where A3 is the only thing that helps.

Feasibility. Addressable from 20-s telemetry. Assumes in-place resize or cheap restart; ASI's actual mechanism is unknown.

E5. Explicit, segment-aware safety constraints A+B

Hypothesis. (H9) A per-segment risk budget (e.g. 1–2% for GenAI/production, 5–8% for rec/non-production) reclaims more GPU-hours than a uniform 5% at equal or lower *expected harm*, where harm weights an under-provisioned production pod higher. GPU memory must be treated as a hard constraint (max/P99) and compute as a soft one (P90).

Design. Define harm $H = \sum_s w_s U_s n_s$ with w_s set by priority. Solve for per-segment margins that maximize R subject to $H \leq H_{\text{uniform 5\%}}$. Separate memory and compute quantile targets.

Metric & decision rule. R at matched H; also report worst-cluster U to show tail safety.

Why it matters. Translates "5% safety constraint" into an operator-facing knob tied to business priority.

Feasibility. Addressable. Harm weights are an assumption; report a sensitivity sweep.

E6. Hybrid policy A

Hypothesis. (H10) Using A1 for warm pods and A3 for cold-start pods matches A1 where it is strong and adds A3's 8.1-point gain on rec-no-history.

Design. Route by history availability; same LOCO protocol.

Metric & decision rule. R at $U = 5\%$ vs. both parents; Wilcoxon over clusters.

Why it matters. Deployable, interpretable, and consistent with the NO-GO finding.

Feasibility. Addressable immediately.

E7. Finer slice sizes (what-if) A

Hypothesis. Adding sizes between 0.5 and 1.0 GPU (e.g. MIG-like sevenths, or 0.6/0.75/0.8/0.9) unlocks GenAI and large-rec reclaim currently blocked by rounding (fig. 5).

Design. Re-round A1/A3 predictions to alternative grids; recompute R, U.

Metric & decision rule. Reclaim by grid; marginal gain per added size.

Why it matters. Quantifies the value of hardware partitioning (MIG profiles) vs. better prediction.

Feasibility. Upper bound only: assumes the slice is actually enforceable and isolated.

E8. Per-cluster recalibration A, B

Hypothesis. Most of B's unseen-cluster calibration error is base-rate shift (35–67% failure by cluster), so a prior-shift correction from a small labelled sample (first day) restores ECE $\leq$0.06; weighted conformal restores A's coverage to $\approx$95%.

Design. Label-shift adjustment of B's logits using day-1 base rate; covariate-weighted split conformal for A. Robustness check: replicate Zhang and Liao's machine-grouped 5-fold protocol [31] on D2 to show how much of the AUC gap (0.82 vs. 0.65–0.75) is due to evaluation protocol.

Metric & decision rule. Unseen-cluster ECE and coverage before/after.

Why it matters. Commercial systems recalibrate per region/cluster continuously; this is the cheapest fix for the transfer gap.

Feasibility. Addressable.

E9. Leakage-free history and peak-aware labels A

Hypothesis. Rebuilding `wl_hist_*` from pods *finished* before admission removes the 8% optimistic bias; P99/max memory labels raise required slices modestly but make U meaningful.

Design. SQL rebuild; re-run the full gate on fixed data before any other experiment is trusted.

Metric & decision rule. Change in pinball, R, U vs. table 5.

Why it matters. Every other result depends on this.

Feasibility. Addressable; highest priority.

E10. Stretch: transfer and simulation A+B

Hypothesis. Findings transfer to a second trace (Tier 2, xMaaS) and survive a scheduler replay (Tier 3) where shrunk pods are actually placed.

Design. Replay the test window in a simple discrete-event simulator with predicted slices; measure placement failures, queueing delay and GPUs freed.

Metric & decision rule. Simulated R vs. trace-estimated R.

Why it matters. Closes the counterfactual gap that Model B cannot.

Feasibility. Partly addressable; simulator fidelity is the main threat.

10.3 Experiment summary

11 Limitations

- **No latency data.** U is the only safety proxy; the trace has no request-latency or SLO-violation field. We cannot claim SLA neutrality, only that predicted slices cover observed busy-hour demand.

Table 16: Experiments by priority. Gap type refers to table 12.

ID	Topic	Model	Gap type	Priority
E9	Leakage-free history, P99 labels	A	Modelling	1
E6	Hybrid policy	A	Modelling	1
E1	Workload mixing (rec vs. GenAI)	A, B	Feature eng.	2
E2	Workload–GPU compatibility	B	Feature eng.	2
E3	Production vs. non-production	A, B	Feature eng.	2
E4	Runtime patterns / resize	A	Feature eng.	2
E8	Per-cluster recalibration	A, B	Modelling	2
E5	Segment-aware safety budgets	A+B	Modelling	3
E7	Finer slices (what-if)	A	Inherent (bounded)	3
E10	Transfer + simulator	A+B	Inherent (partly)	stretch

- **Labels are P90s of hourly averages.** Telemetry is sampled every 20 seconds, so true peaks are higher. The GPU-memory term should use a maximum or P99.
- **Possible leakage** in history features (table 10).
- **Reclaiming needs a mechanism.** The operator reports GPU sharing is rarely used because isolation is weak, and its own SpotGPU mechanism already raises allocation from 68% to 93%. How reclaimed slices would be consumed is open.
- **Counterfactual blindness.** The trace records what was allocated, not what would have happened at a smaller size; Model B extrapolates.
- **External validity.** One operator, one six-month window, 10 usable held-out clusters; anonymized model identities.

12 Plan and Timeline

Phases 0–5 of the original plan (data, datasets, baselines, Model A, Model B, composition) are complete; interpretation is done for Model A. Figure 9 lays out the remaining work.

13 Conclusion

The headline is a useful negative result wrapped around a positive one. *Positive:* at a 5% under-provisioning budget, roughly a fifth to a quarter of reserved online-inference GPU-hours are reclaimable on clusters never seen in training, concentrated in recommendation pods with history. *Negative:* a learned model does not beat a per-workload history rule on accuracy; most of the value is reachable with a rule an operator could deploy this week, and ML's marginal value is confined to cold-start pods. *Open:* the placement-risk side, because the trace barely contains the fractional allocations a right-sizing policy would create.

Compared to commercial schedulers, this gap is expected rather than embarrassing: production systems own the actuation path (MIG, MPS, time-slicing, live resize), observe latency, and can run controlled experiments, while a trace-based study can only estimate the *opportunity*. The next round of work targets the gaps that modelling can close (leakage, labels, workload-mixing and runtime features, per-cluster calibration, segment-specific safety budgets) and documents honestly the ones it cannot.

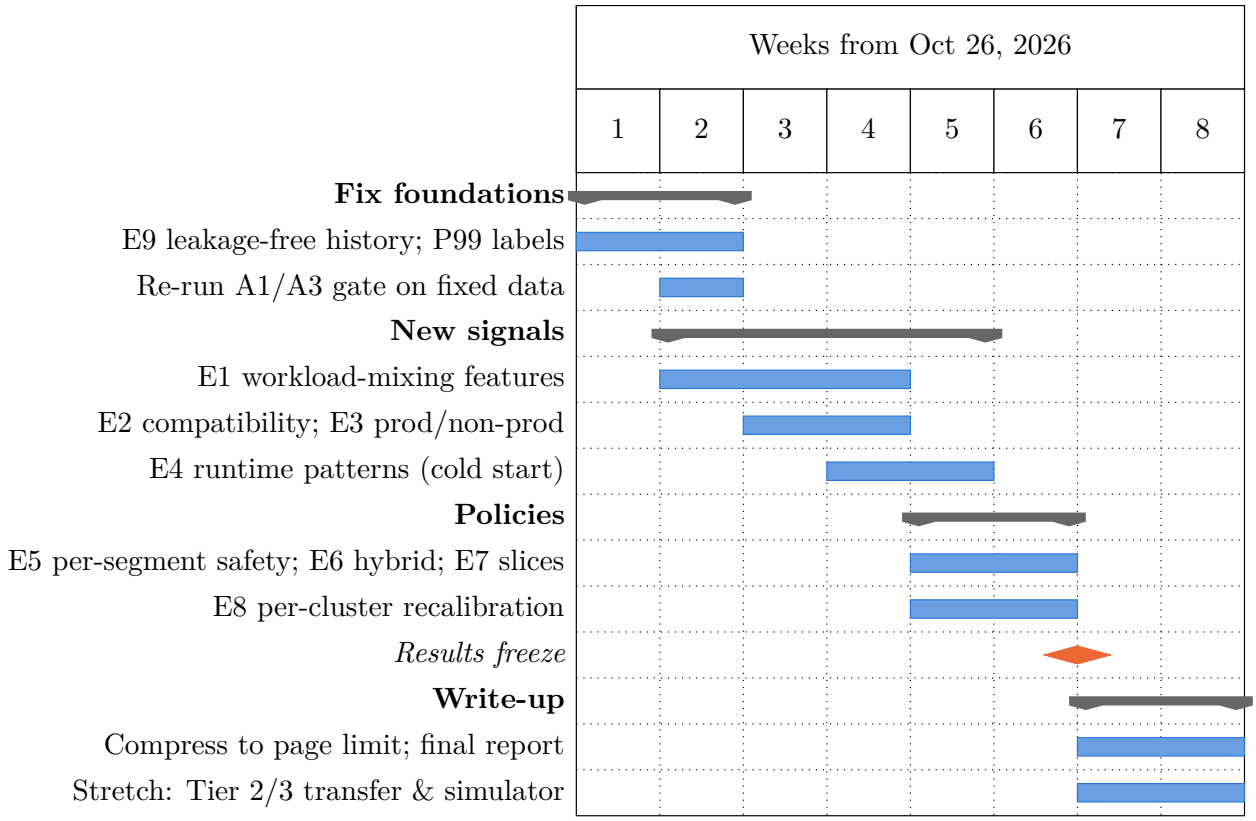


Figure 9: Remaining work. Stretch items are cut first if time runs short.

References

[1] Amazon Web Services. Reduce model deployment costs by 50% on average using SageMaker's latest features. https://aws.amazon.com/blogs/machine-learning/reduce-model-deployment-costs-by-50-on-average-using-sagemakers-latest-features/, 2023.

[2] Amazon Web Services. Announcing up to 45% price reduction for Amazon EC2 NVIDIA GPU-accelerated instances. https://aws.amazon.com/blogs/aws/announcing-up-to-45-price-reduction-for-amazon-ec2-nvidia-gpu-accelerated-instances/, 2025.

[3] Amazon Web Services. Use time-slicing / MIG with NVIDIA GPUs on Amazon EKS. https://docs.aws.amazon.com/eks/latest/userguide/device-management-nvidia-time-slicing.html, 2026.

[4] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. arXiv:2107.07511, 2021.

[5] Eli Cortez et al. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2017.

[6] Jiangfei Duan et al. MuxServe: Flexible spatial-temporal multiplexing for multiple LLM serving. In *International Conference on Machine Learning (ICML)*, 2024.

[7] Yanjie Gao et al. An empirical study on low GPU utilization of deep learning jobs. In *International Conference on Software Engineering (ICSE)*, 2024.

[8] Google Cloud. About GPU sharing strategies in GKE. https://docs.cloud.google.com/kubernetes-engine/docs/concepts/timesharing-gpus, 2026.

[9] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[10] Qinghao Hu et al. Characterization and prediction of deep learning workloads in large-scale GPU datacenters. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[11] Myeongjae Jeon et al. Analysis of large-scale multi-tenant GPU clusters for DNN training workloads. In *USENIX Annual Technical Conference (ATC)*, 2019. URL https://www.usenix.org/conference/atc19/presentation/jeon.

[12] Suyi Li, Lingyun Yang, et al. Heterogeneity at hyperscale: Characterization and scheduling of large production AI clusters at Alibaba. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026. URL https://www.usenix.org/conference/osdi26/presentation/li-suyi.

[13] Microsoft. Create a multi-instance GPU node pool in Azure Kubernetes Service. https://learn.microsoft.com/en-us/azure/aks/gpu-multi-instance, 2026.

[14] Deepak Narayanan et al. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

[15] NVIDIA. Introducing NVIDIA Dynamo: A low-latency distributed inference framework for scaling reasoning AI models. NVIDIA Developer Blog, 2025.

[16] NVIDIA. MIG user guide: Supported MIG profiles. https://docs.nvidia.com/datacenter/tesla/mig-user-guide/latest/supported-mig-profiles.html, 2026.

[17] Zhicheng Pan et al. MagicScaler: Uncertainty-aware, predictive autoscaling. *Proceedings of the VLDB Endowment*, 16(12), 2023.

[18] Yaniv Romano, Evan Patterson, and Emmanuel Candès. Conformalized quantile regression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[19] Krzysztof Rzadca et al. Autopilot: Workload autoscaling at Google scale. In *European Conference on Computer Systems (EuroSys)*, 2020. URL https://research.google/pubs/pub49174/.

[20] B. Sharma, L. Yang, H. Pham, and Z. Qin. Multi-tenancy for AI inference at Meta scale. https://atscaleconference.com/multi-tenancy-for-ai-inference-at-meta-scale/, 2023.

[21] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. USHER: Holistic interference avoidance for resource optimized ML inference. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[22] Dharma Shukla et al. Singularity: Planet-scale, preemptive and elastic scheduling of AI workloads. arXiv:2202.07848, 2022. URL https://arxiv.org/abs/2202.07848.

[23] Foteini Strati, Xianzhe Ma, and Ana Klimovic. Orion: Interference-aware, fine-grained GPU sharing for ML applications. In *European Conference on Computer Systems (EuroSys)*, 2024. doi: 10.1145/3627703.3629578.

[24] Qizhen Weng et al. MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2022. URL https://www.usenix.org/conference/nsdi22/presentation/weng.

[25] Qizhen Weng et al. Beware of fragmentation: Scheduling GPU-sharing workloads with fragmentation gradient descent. In *USENIX Annual Technical Conference (ATC)*, 2023. URL https://www.usenix.org/conference/atc23/presentation/weng.

[26] Yuxing Xiang, Xue Li, Kun Qian, et al. Aegaeon: Effective GPU pooling for concurrent LLM serving on the market. In *ACM Symposium on Operating Systems Principles (SOSP)*, pages 1030–1045, 2025.

[27] Wencong Xiao et al. Gandiva: Introspective cluster scheduling for deep learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018. URL https://www.usenix.org/conference/osdi18/presentation/xiao.

[28] Wencong Xiao et al. AntMan: Dynamic scaling on GPU clusters for deep learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020. URL https://www.usenix.org/conference/osdi20/presentation/xiao.

[29] Fei Xu et al. iGniter: Interference-aware GPU resource provisioning for predictable DNN inference in the cloud. arXiv:2211.01713 (IEEE TPDS), 2022.

[30] Lingyun Yang et al. GPU-disaggregated serving for deep learning recommendation models at scale. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2025. URL https://www.usenix.org/conference/nsdi25/presentation/yang.

[31] Tao Zhang and Bin Liao. Ineffective occupancy in heterogeneous GPU clusters: Prediction, contribution analysis, and readiness-aware scheduling. arXiv:2610.25066, 2026. URL https://arxiv.org/abs/2610.25066.

A Research Trail (Chronological)

When	Step
Early Sep 2026	Topic chosen: can GPU-trace patterns raise utilization without compromising inference latency/SLA? Motivation via utilization identity (eq. (1)). Dataset: ASI trace. Open items: schema, sampling, scope, compute budget.
Mid Sep 2026	1–2% sample used for first visualizations. Scope narrowed to rec + GenAI online inference (supervisor comment). References gathered: OSDI '26 ASI, Philly traces, Resource Central.
Late Sep 2026	Day/hour recovery; D1 and D2 built; plan pre-registered (models, splits, LOCO, conformal, go/no-go gate).
Oct 4, 2026	Progress report: Phases 0–5 done. Model A NO-GO on accuracy gate; ≈21% reclaimable at $U = 5\%$; Model B AUC 0.75/0.65; A+B composition as what-if.
Oct 26, 2026	Reference audit: the OSDI '26 PDF link in the early notes (`osdi26-wu-tianyuan.pdf`) is the WEAVE RL post-training paper; the ASI trace paper is Li et al. [12]. Closest prior work to Model B found [31].
Oct 26, 2026	This proposal: industry comparison, gap taxonomy, business value, next experiments (E1–E10).

B Reproducibility

All figures are regenerated by `scripts/make_figures.py` from the numbers reported in section 6; no figure in this document reports a PROPOSED experiment. The proposal compiles with `tectonic main.tex`.